SIMATS ENGINEERING

ASSESSMENT TOOL

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: Deign Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:25**

Duration : 1 Hour

AT2 - Design Task

Code of Conduct:

I **J. SAI DHEERAJ (192311191)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

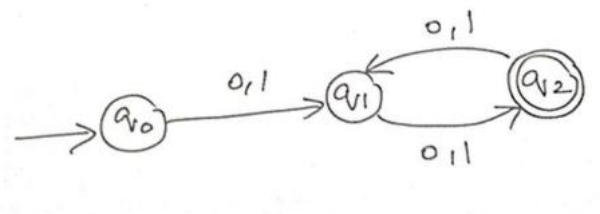
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **J. SAI DHEERAJ**

SIMATS ENGINEERING

ASSESSMENT TOOL

1.Design the regular expression for the language represented by the given automata



2.Consider the following grammar

S->aSc|T|U|e

T-> bTc|e

U->aUb|e

A} What is the language defined by the grammar?

B)Give the derivation tree for the word aaabbc

C)A Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

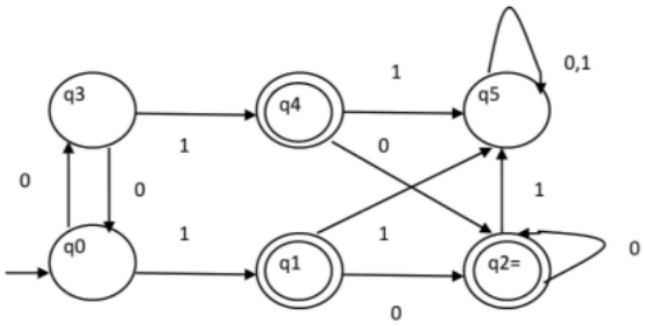
3. Design a non deterministic finite automata with ε-transitions to represent the language denoted by the regular expression

3. Design a non deterministic finite automata with ε-transitions to represent the language denoted by the regular expression
00*1+11*0

4. **I)**Show that if L1 and L2 are regular then (L1+l2) also regular

Ii)Is the language { a^p| p is prime} regular? Justify

5. Design minimized automata, Consider the DFA given below. Find the states that are equivalent and construct a minimum state automata equivalent to the given automata. Draw the minimized automata, write its transition table and mark initial and final states



1. Design the regular expression for the given automaton

From the diagram:

- Initial state: q_0
- Final state: q_2
- $q_0 \xrightarrow{0,1} q_1$
- $q_1 \xrightarrow{0,1} q_2$
- $q_2 \xrightarrow{0,1} q_1$

To reach the final state, the automaton must read at least two symbols. After that, it can read any number of additional symbols because it alternates between q_1 and q_2.

Regular expression:

$$\boxed{(0+1)(0+1)((0+1)(0+1))^*}$$

Equivalent simplified expression:

$$\boxed{(0+1)(0+1)(0+1)^*}$$

This represents all binary strings of length two or more.

2. Consider the following grammar

Given:

$$\begin{aligned} S &\rightarrow aSc \mid T \mid U \mid \epsilon\\ T &\rightarrow bTc \mid \epsilon\\ U &\rightarrow aUb \mid \epsilon \end{aligned}$$

A) What is the language defined by the grammar?

- $S \rightarrow aSc$ adds one a at the beginning and one c at the end.
- $T \rightarrow bTc$ generates equal numbers of b and c.
- $U \rightarrow aUb$ generates equal numbers of a and b.
- ϵ represents the empty string.

Therefore, the language is:

$$\boxed{L(S)=\{a^n x c^n\mid x\in L(T)\cup L(U)\cup\{\epsilon\},\ n\geq0\}}$$

where

$$L(T)=\{b^n c^n\mid n\geq0\}$$

and

$$L(U)=\{a^n b^n\mid n\geq0\}$$

B) Give the derivation tree for the word aaabbc

Using the production \(S\rightarrow aSc\), the derivation begins:

\[S\Rightarrow aSc \Rightarrow aaScc \]

However, the word aaabbc ends with only one c and cannot be generated by the grammar. The grammar's recursive \(S\rightarrow aSc\) production adds matching numbers of a and c, and the remaining productions cannot remove those unmatched c symbols.

Answer: aaabbc is not generated by the given grammar, so no valid derivation tree exists.

C) Is the grammar ambiguous? Justify.

Yes, the grammar is ambiguous.

The string \(\epsilon\) has multiple distinct derivations:

Derivation 1:

\[S\Rightarrow\epsilon \]

Derivation 2:

\[S\Rightarrow T\Rightarrow\epsilon \]

Derivation 3:

\[S\Rightarrow U\Rightarrow\epsilon \]

Since the same string has multiple distinct derivations, the grammar is ambiguous.

3. Design an NFA with \(\epsilon\)-transitions for the regular expression

Given regular expression:

\[\boxed{00^*1+11^*0} \]

The + represents union (OR).

- \(00^*1\): one or more zeros followed by one 1.
- \(11^*0\): one or more ones followed by one 0.

NFA transition diagram

Branch 1: \(00^*1\)

\(q_0\)

\(q_1\)

\(q_2\)

\(q_f\)

Start → ε to branch start; labels: 0, 0 (loop on q_2), 1. q_f is final.

Branch 2: \(11^*0\)

\(q_0\)

\(q_3\)

\(q_4\)

\(q_f\)

Start → ε to branch start; labels: 1, 1 (loop on q_4), 0. q_f is final.

Transition table

State	Input 0	Input 1	Input \(\epsilon\)
\(q_0\)	—	—	\(\{q_1,q_3\}\)
\(q_1\)	\(q_2\)	—	—
\(q_2\)	\(q_2\)	\(q_f\)	—
\(q_3\)	—	\(q_4\)	—
\(q_4\)	\(q_f\)	\(q_4\)	—
\(q_f\)	—	—	—

- Initial state: \(q_0\)
- Final state: \(q_f\)

The \(\epsilon\)-transitions allow the automaton to choose either branch without consuming an input symbol.

4. Regular languages

i) Show that if \(L_1\) and \(L_2\) are regular, then \(L_1\cup L_2\) is also regular.

Proof:

Since \(L_1\) and \(L_2\) are regular, there exist finite automata \(M_1\) and \(M_2\) that accept them.

Construct a new NFA by:

1. Creating a new initial state \(q_s\).
2. Adding \(\epsilon\)-transitions from \(q_s\) to the initial states of \(M_1\) and \(M_2\).
3. Keeping the final states of both automata as final states.

The new automaton accepts a string if it belongs to either \(L_1\) or \(L_2\).

Therefore,

\[\boxed{L_1\cup L_2\text{ is regular}} \]

Hence proved.

ii) Is $L=\{a^p\mid p\text{ is prime}\}$ regular? Justify.

Answer: No, the language is not regular.

Proof using the pumping lemma:

Assume that L is regular. Let n be its pumping length.

Choose a prime number $p>n$ and the string

$$ w=a^p $$

According to the pumping lemma, $w=xyz$, where:

- $|xy|\leq n$
- $|y|>0$
- $xy^iz\in L$ for every $i\geq0$

Let $y=a^k$, where $k>0$. Pumping y with $i=p+1$ produces a string of length

$$ p+pk=p(1+k) $$

Since p is prime and $1+k>1$, this length is composite. Therefore, the resulting string does not belong to L.

This contradicts the pumping lemma.

$$ \boxed{L=\{a^p\mid p\text{ is prime}\}\text{ is not regular}} $$

5. Minimize the given DFA

From the given diagram, the states are q_0,q_1,q_2,q_3,q_4,q_5.

- Initial state: q_0
- Final states: q_1,q_2,q_4

Step 1: Identify equivalent states

The equivalent states are:

$$ \boxed{q_0\equiv q_3} $$

Both are non-final states; on input 0, they transition within the same pair, and on input 1, they transition to a final state.

Also,

$$ \boxed{q_1\equiv q_2\equiv q_4} $$

These are final states, each transitioning to a final state on 0 and to q_5 on 1.

State q_5 is a non-final dead state that loops on both inputs.

Thus, the minimized DFA contains three states:

$$ A=\{q_0,q_3\},\quad B=\{q_1,q_2,q_4\},\quad C=\{q_5\} $$

Step 2: Transition table of the minimized DFA

State	Input 0	Input 1
$\rightarrow A$	A	B
$*B$	B	C
C	C	C

Here, $\rightarrow$ denotes the initial state and $*$ denotes a final state.

Step 3: Minimized automaton diagram

A is the initial state; B is the only final state; C is the dead state.

Step 4: Conclusion

The original DFA has six states, while the minimized DFA has only three states.

$$ \boxed{\text{Minimum number of states}=3} $$

State minimization reduces memory requirements and simplifies the transition logic while preserving the language accepted by the original DFA.